Week 3: Git and GitHub

AI Development Course

Installs and git steps tested with git 2.43.0 and gh 2.102.0 on Ubuntu Linux (in a container), 12 October 2026. Everything that needs a GitHub account, and every Mac step, is being tested on real Mac and Windows laptops.

Today

- What git and GitHub are, and why they matter with AI
- Setup: a GitHub account, git, the GitHub CLI (`gh`), signing in
- Live demo: a repo, a private GitHub copy, a pull request
- Undo safely
- Inviting your instructor
- The assignment, and next week

Git: snapshots and history

- **Git** saves **snapshots** of the files in a folder
- Each snapshot is a **commit**: what every file looked like, plus a message
- The list of commits is your **history**
- You can always go back to an earlier snapshot

A folder git looks after is a **repository** (**repo**).

Branches and pull requests

- A **branch** is a **safe draft**. `main` is the official version.
- Try a change on a branch; `main` stays exactly as it was
- A **pull request** (PR): "review this before it becomes official"
- Read what changed, then **merge** it into `main`

GitHub: the online copy

- **github.com** keeps an online copy of your repo: your backup
- **push**: send new commits up to GitHub
- **pull**: bring new commits down from GitHub
- Your repos this term are **private**: only you, and people you invite

Your business ideas stay yours.

Why this matters with AI

Claude Code changes **real files** on your laptop.

Sometimes it changes something you didn't expect.

Git is your **undo button**:

commit before you ask for a big change.

How you'll work this week

You ask Claude Code in **plain English**. For each step the README shows:

1. **The prompt** to type
2. **What Claude Code will probably do:** the commands, the permission prompts
3. **What to check** afterwards

Runs without asking (read-only)	Asks first
`git status`, `git log`, `git diff`	`git add`, `git commit`, `git push`, `git restore`, `git revert`

Use **manual mode**: `Shift + Tab` until `manual mode on`.

Step 1: a GitHub account (being tested)

- Sign up at https://github.com/signup, verify your email
- **Settings > Emails**:
  - tick **Keep my email addresses private**
  - tick **Block command line pushes that expose my email**
- Copy your private address:

```
12345678+yourusername@users.noreply.github.com
```

Step 2: check git, tell it who you are

In your terminal:

```
git --version
```

```
git version 2.43.0
```

Ubuntu says git is not found? `sudo apt update && sudo apt install -y git`

```
git config --global user.name "Your Name"
git config --global user.email "12345678+yourusername@users.noreply.github.com"
```

Mac (being tested): a window may offer the "command line developer tools". Click Install, then run `git --version` again.

Step 3: install `gh`, GitHub's command-line tool

Mac (Terminal), being tested	Windows (Ubuntu window)
Install Homebrew: the line from https://brew.sh	Paste GitHub's official apt block (in the README)
Return to continue; your Mac password (shows nothing)	Your Ubuntu password (shows nothing)
Run the **Next steps** lines it prints	
Close and reopen Terminal	
`brew install gh`	

Check, in either: `gh --version`

```
gh version 2.102.0 (2026-09-30)
```

Mac: macOS 15 (Sequoia) or later. Intel Macs may work but aren't officially supported by Homebrew.

Step 4: sign in with `gh` (being tested)

```
gh auth login
```

It asks about	Choose
where you use GitHub	**GitHub.com**
protocol for Git operations	**HTTPS**
authenticate Git with your GitHub credentials	**Y**
how to authenticate	**Login with a web browser**

Type the one-time code in the browser. WSL: open https://github.com/login/device yourself.

Then check: `gh auth status` shows **your** username. No tokens, ever.

Live demo: make a repo

```
cd ~/my-first-project && claude
```

```
Make this folder a git repository with the branch name main.
Then save a first snapshot (a commit) of all the files, with
the message "First snapshot". Explain each command first.
```

Probably: `git init -b main` (or `git init` + `git branch -M main`), `git add`, `git commit`

Check: "Show me the git history and the current branch"

```
74659dd Add README
main
```

Live demo: a private GitHub copy (being tested)

```
Use gh to create a new PRIVATE GitHub repository called
my-first-project from this folder, with the remote named
origin, and push the main branch. It must be private.
```

Probably:

```
gh repo create my-first-project --private --source=. --remote=origin --push
```

Before you say Yes, look for `--private`. No `--private`? Say No.

The loop you'll use all term

branch -> change -> push -> pull request -> merge -> pull

```
Make a new branch called add-goals. On it, create goals.md
with three goals I have for this course, commit it, push the
branch to GitHub, and open a pull request with a clear title
and a short description of what changed and why.
```

Probably: `git switch -c add-goals`, `git add`, `git commit`, `git push -u origin add-goals`, `gh pr create`

Merge on GitHub, then pull (being tested)

1. Open the PR's address. Read **Files changed**.
2. **Merge pull request**, then **Confirm merge**
3. Back in Claude Code: "Switch back to main and pull the latest changes from GitHub."

Pull after you merge. Forget, and your next push is refused:

```
! [rejected]        main -> main (fetch first)
```

Fix: ask Claude Code to pull **as a merge** (`git pull --no-rebase --no-edit`), then push again. No reset, no force-push.

A prompt with `--force` or `reset --hard`? Say **No**.

Undo safely

You want to	Ask for	Result
throw away a change you haven't committed	`git restore`	file back to the last snapshot; **that change is gone for good**
undo a commit, even a pushed one	`git revert`	a new commit that does the opposite; history keeps both

```
9fe6f0b Revert "Add intro line"
2be7a5d Add intro line
```

Never `git reset --hard` or `git push --force` on pushed work. See it in a prompt? Say **No**.

Invite your instructor (being tested)

Your repo is private, so invite your instructor. On github.com:

1. Your repo, then **Settings**
2. **Collaborators**, then **Add people**
3. Type `vasuchettyphd`, select it
4. **Add vasuchettyphd to my-first-project**

Your instructor accepts from the invitation email. **Don't make the repo public.**

Assignment

One document with:

- Your **private** repo's address, with `vasuchettyphd` added
- A screenshot of the repo page showing **Private**
- The address of a **merged** pull request
- One paragraph: when would you use a branch?

Criterion	Points
Repo exists, screenshot shows Private (2), collaborator added (2)	4
Merged PR with a sensible title and description	3
Reflection: a real branch situation, the risk, a safe undo	3

Next week: Planning a project

You choose the tool you'll build this term.

Its history will live in a repo like the one you made today.